Git Cherry-Pick and Interactive Rebase Tutorial

Date: 2026-04-04
Task: Create tutorial documentation for Git cherry-pick and interactive rebase operations

---

Table of Contents

Prerequisites

Before proceeding with either operation, ensure you:

- ✅ Have a clean working directory (`git status` shows no uncommitted changes)
- ✅ Have committed or stashed any uncommitted work
- ✅ Understand that both operations rewrite Git history
- ✅ Have pushed/fetched the latest changes from remote

```
# Check current status
git status

# Check available branches
git branch -a

# Ensure you're on the correct branch
git branch --show-current
```

Understanding the Scenario

Visual Representation

```
feat-A branch:          master branch:
A1 ── B1 ── C1 ── D1    M1 ── M2 ── M3
                    ↑
                (behind feat-A)
```

- **feat-A**: Feature branch with commits A1, B1, C1, D1
- **master**: Main branch with commits M1, M2, M3

When to Use Each Operation

Operation	Use Case	Effect
Cherry-Pick	Apply specific commits from one branch to another	Creates new commits with different hashes
Interactive Rebase	Clean up, reorder, or combine commits before merging	Rewrites commit history

---

Cherry-Picking Commits

What is Cherry-Picking?

Cherry-picking applies a specific commit (or commits) from one branch to another. It creates a **new commit** with a **different hash** but the same changes.

Step-by-Step: Cherry-Pick from feat-A to master

Step 1: Find the Commits to Cherry-Pick

```
# View commit history on feat-A branch
git log feat-A --oneline

# Example output:
# d4e5f6d (feat-A) D1 - Fourth feature commit
# c3b4a5c C1 - Third feature commit
# b2c3d4b B1 - Second feature commit
# a1b2c3a A1 - First feature commit
```

Step 2: Switch to the Target Branch (master)

```
# Switch to master
git checkout master

# Or using newer syntax
git switch master

# Pull latest changes from remote
git pull origin master
```

Step 3: Cherry-Pick the Commit(s)

Single Commit:

```
# Cherry-pick a single commit
git cherry-pick c3b4a5c

# Result: New commit created on master with same changes as C1
```

Multiple Commits:

```
# Cherry-pick multiple commits (inclusive range)
git cherry-pick a1b2c3a^..c3b4a5c

# Or specify individual commits
git cherry-pick a1b2c3a b2c3d4b c3b4a5c
```

Without Committing (Edit the changes):

```
# Cherry-pick but don't commit automatically
git cherry-pick c3b4a5c --no-commit

# Make additional changes if needed
git add .
git commit -m "Your custom commit message"
```

Step 4: Handle Conflicts (if any)

If cherry-pick causes conflicts:

```
# Git will pause and indicate conflicts
# Edit conflicted files to resolve

# After resolving conflicts
git add <resolved-files>

# Continue cherry-pick
git cherry-pick --continue

# Or abort if you want to cancel
git cherry-pick --abort
```

Step 5: Push to Remote

```
# Push the cherry-picked commits
git push origin master
```

Cherry-Pick Results

```
Before:                      After cherry-pick C1:
feat-A:          master:     feat-A:          master:
A1—B1—C1—D1      M1—M2—M3    A1—B1—C1—D1      M1—M2—M3—C1'
                                                (new commit hash)
```

Interactive Rebase

What is Interactive Rebase?

Interactive rebase allows you to **rewrite, reorder, edit, squash, or drop commits** in your history. It's commonly used to clean up a feature branch before merging.

Step-by-Step: Interactive Rebase on master from feat-A

Scenario: Rebase master onto feat-A

This will **replay all master commits on top of feat-A**:

```
Before rebase:                After rebase:
feat-A:          master:      feat-A:           master:
A1—B1—C1—D1      M1—M2—M3      A1—B1—C1—D1—M1'—M2'—M3'
                                                (master replayed on feat-
A)
```

Step 1: Ensure Clean State

```
# Check for uncommitted changes
git status

# Stash any uncommitted work
git stash push -m "Temporary work"

# Switch to master
git checkout master
```

Step 2: Start Interactive Rebase

```
# Rebase master onto feat-A
# This replays master's commits on top of feat-A
git rebase -i feat-A

# Or rebase current branch (master) onto feat-A
git rebase -i feat-A
```

Note: The command `git rebase -i feat-A` while on master means: > "Take all commits on master that are not in feat-A, and replay them on top of feat-A"

Step 3: The Interactive Rebase Editor

You'll see a text editor with commits listed:

```
pick M3' M3 commit message
pick M2' M2 commit message
pick M1' M1 commit message

# Rebase instructions:
# p, pick = keep commit as is
# r, reword = keep commit but edit message
# e, edit = pause for amending
# s, squash = merge with previous commit
# f, fixup = merge with previous, discard message
# d, drop = remove commit
```

Available Commands:

Command	Action
`pick`	Keep commit as-is (default)
`reword`	Edit commit message
`edit`	Pause for amending the commit
`squash`	Combine with previous commit
`fixup`	Combine with previous, discard this message
`drop`	Remove the commit

Step 4: Edit and Save

Make your changes, then save and close the editor.

Example: Reorder commits

```
pick M2' M2 commit message  # Moved up
pick M3' M3 commit message  # Moved down
pick M1' M1 commit message  # Moved to bottom
```

Example: Squash commits

```
pick M1' M1 commit message
squash M2' M2 commit message  # Will be combined into M1'
pick M3' M3 commit message
```

Step 5: Handle Conflicts (if any)

```
# If conflicts occur during rebase:
# 1. Resolve conflicts in files
# 2. Stage resolved files
git add <resolved-files>

# 3. Continue rebase
git rebase --continue

# OR skip this commit
git rebase --skip

# OR abort the entire rebase
git rebase --abort
```

Step 6: Force Push (if needed)

⚠️ **Warning:** Rebase rewrites history. If master was already pushed, you'll need to force push:

```
# Force push with lease (safer)
git push origin master --force-with-lease
```

```
# Regular force push (dangerous if others are working on the
        branch)
git push origin master --force
```

Alternative: Rebase feat-A onto master

If you want to update feat-A with master changes first:

```
# Switch to feat-A
git checkout feat-A

# Rebase feat-A onto master (feat-A commits replayed on master)
git rebase master

# Result: M1—M2—M3—A1'—B1'—C1'—D1'
```

Common Pitfalls and Solutions

1. Merge Conflicts During Cherry-Pick

Problem: The same file was modified differently in both branches.

```
# Solution: Resolve manually
git status                    # See conflicts
edit conflicted files
git add <resolved-files>
git cherry-pick --continue
```

2. Empty Cherry-Pick (Already Applied)

Problem: Commit was already applied to target branch.

```
# Solution: Skip or use --empty flag
git cherry-pick --skip
# or
git cherry-pick --allow-empty
```

3. Rebase Conflicts

Problem: Multiple conflicts during rebase.

```
# Solution: Resolve one at a time
git rebase --continue   # After each resolution
# or abort everything
git rebase --abort
```

4. Lost Commits After Rebase

Problem: Commits seem disappeared after aborted rebase.

```
# Solution: Use reflog to recover
git reflog
# Find the commit hash before rebase
git reset --hard <commit-hash>
```

5. Detached HEAD State

Problem: You're in detached HEAD mode.

```
# Check status
git status

# Solution: Create a branch from current state
git checkout -b new-branch

# Or return to your branch
git checkout master
```

Best Practices

Cherry-Pick Best Practices

1. **Use for hotfixes**: Cherry-pick is ideal for applying critical fixes from feature branches to release branches
2. **Keep it minimal**: Cherry-pick as few commits as necessary
3. **Document clearly**: Add "cherry-picked from commit " to commit messages
4. **Test thoroughly**: Cherry-picked commits behave differently in new context

```
# Good: Single commit cherry-pick
git cherry-pick abc1234

# Acceptable: Related commits
git cherry-pick abc1234 def5678

# Avoid: Many unrelated commits (merge instead)
```

Interactive Rebase Best Practices

1. **Never rebase public history**: Only rebase local or unpushed commits
2. **Use on feature branches**: Keep master/main history clean
3. **Clean before merge**: Rebase feature branch onto master before merging PR

4. **Squash related work**: Combine small “fix typo” commits into feature commit
5. **Write clear messages**: Reword to create meaningful commit history

```
# Good workflow:
git checkout feat-A
git rebase master            # Update with latest master
git rebase -i HEAD~5         # Clean up last 5 commits
git checkout master
git merge feat-A             # Fast-forward merge
```

Safety Checks

```
# Check what will be rebased (dry run)
git rebase -i --dry-run feat-A

# See differences before and after
git diff master feat-A

# Check if commits are in both branches
git log master..feat-A       # Commits in feat-A not in master
git log feat-A..master       # Commits in master not in feat-A
```

Recovery Commands

```
# Undo local changes (keep commits)
git rebase --abort

# Undo everything including changes
git reset --hard ORIG_HEAD

# Find lost commits
git reflog

# Create branch from previous state
git branch backup-branch ORIG_HEAD
```

Quick Reference Commands

Cherry-Pick

```
# Basic cherry-pick
git cherry-pick <commit-hash>

# Multiple commits
git cherry-pick <hash1> <hash2> <hash3>

# Range of commits
```

```
git cherry-pick <start>^..<end>

# Without committing
git cherry-pick <hash> --no-commit

# Continue after conflicts
git cherry-pick --continue

# Abort cherry-pick
git cherry-pick --abort
```

Interactive Rebase

```
# Rebase current branch onto another
git rebase -i <target-branch>

# Rebase last N commits
git rebase -i HEAD~<n>

# Rebase from specific commit
git rebase -i <commit-hash>

# Continue after conflicts
git rebase --continue

# Skip current commit
git rebase --skip

# Abort rebase
git rebase --abort

# Show original branches
git rebase --show-orig-ref
```

Summary

Aspect	Cherry-Pick	Interactive Rebase
Purpose	Apply specific commits	Rewrite/reorganize history
New Hashes?	Yes	Yes
Use Case	Hotfixes, selective backports	Clean up before merge
Scope	Individual commits	Branch history
Danger Level	Medium (local changes only)	High (rewrites history)

Aspect	Cherry-Pick	Interactive Rebase
Undo	Reset/delete new commits	`git rebase --abort` or reflog

Additional Resources

- Git Cherry-Pick Documentation
- Git Rebase Documentation
- Interactive Rebase Guide

Implementation Plan: Created comprehensive tutorial documentation covering both cherry-pick and interactive rebase operations with step-by-step instructions, examples, and best practices.

Change Log: Created new tutorial document in notes folder.